INF560 / Evaluación / EP2

EXAMEN DE SEGUNDO PARCIAL · TRABAJO INDEPENDIENTE

TuriPotosí

Plataforma de reservas de tours con Laravel 13

Gestión 02/2026 · Individual · 100 pts · En casa + defensa

GET POST PUT PATCH DELETE métodos HTTP usados en la tabla de rutas

Dato	Registro
Fecha de publicación	Viernes 6 de noviembre de 2026
Fecha de entrega y defensa	Viernes 13 de noviembre de 2026

Docente: M. Sc. Huáscar Fedor Gonzales Guzmán
Universidad Autónoma Tomás Frías · Facultad de Ciencias Puras · Ingeniería Informática

01 Descripción general

La empresa TuriPotosí ofrece recorridos turísticos por la ciudad y sus alrededores: el Cerro Rico, la Casa Nacional de Moneda, las lagunas del Kari Kari, rutas gastronómicas, entre otros. Necesita una aplicación web donde el público consulte el catálogo de tours, los turistas registrados hagan reservas, los guías vean quiénes asistirán a sus recorridos y un administrador gestione todo el contenido.

Usted debe construir esa aplicación de forma individual, en casa, aplicando de manera integrada lo trabajado en laboratorio durante el semestre. El examen no evalúa temas aislados: evalúa su capacidad de combinarlos en un proyecto funcional, ordenado y explicable.

Contenidos que se evalúan

Contenido del curso	Dónde se aplica en el proyecto
Rutas, controladores y vistas Blade	Catálogo público, panel de administración, rutas con nombre y agrupadas
Componentes Blade y Tailwind CSS v4	Layout reutilizable, tarjeta de tour, mensajes flash, diseño responsive
Migraciones, modelos y Eloquent	Cinco tablas propias con claves foráneas, restricciones y casts
Relaciones Eloquent (1:N y N:M)	Categoría–Tour, Guía–Tour, Tour–Servicio, Usuario–Reserva–Tour
Factories y seeders	Base de datos reproducible con migrate:fresh --seed
CRUD con validación	Gestión de tours con Form Requests y mensajes de error
Autenticación	Registro, inicio y cierre de sesión
Autorización: roles y permisos	spatie/laravel-permission, middleware de rol, Policy de reservas
Carga de imágenes	Portada de cada tour en el disco public, con reemplazo y borrado

02 Roles del sistema

La aplicación tiene tres roles. Cada usuario tiene exactamente uno. Quien se registra desde el formulario público recibe automáticamente el rol turista.

Rol	Puede hacer	No puede hacer
`admin`	Crear, editar y eliminar tours; ver todas las reservas y cambiar su estado	Reservar tours
`guia`	Ver los tours que tiene asignados y la lista de reservas de cada uno	Editar tours, reservar, ver reservas de tours ajenos
`turista`	Reservar tours, ver sus reservas y cancelar las que estén pendientes	Acceder al panel admin, ver o cancelar reservas de otros
`visitante`	Ver el catálogo y el detalle de los tours (sin sesión)	Reservar (se le redirige al login)

03 Modelo de datos

Además de la tabla users y las tablas que crean la autenticación, el proyecto debe tener las siguientes tablas. Los nombres de tablas y columnas son obligatorios, porque el docente revisará con las mismas consultas a todos los proyectos.

Tabla	Columnas	Restricciones
categorias	id, nombre, slug, descripcion, timestamps	nombre y slug únicos; descripcion nullable
servicios	id, nombre, timestamps	nombre único
tours	id, categoria_id, guia_id, titulo, slug, descripcion, precio, duracion_horas, cupo_maximo, fecha_salida, imagen, activo, timestamps	FK categoria_id (restrict); FK guia_id → users (nullable, null on delete); slug único; precio decimal(10,2); imagen nullable; activo boolean default true
servicio_tour	servicio_id, tour_id	Tabla pivote N:M; ambas FK con cascade; clave primaria compuesta
reservas	id, user_id, tour_id, cantidad_personas, estado, observacion, timestamps	FK con cascade; estado: pendiente | confirmada | cancelada (default pendiente); observacion nullable

Relaciones que deben existir en los modelos

Modelo	Método	Tipo
Categoria	tours()	hasMany
Tour	categoria(), guia(), servicios(), reservas()	belongsTo, belongsTo (User), belongsToMany, hasMany
Servicio	tours()	belongsToMany
User	reservas(), toursGuiados()	hasMany, hasMany (por guia_id)
Reserva	user(), tour()	belongsTo, belongsTo

> **DEFINICIÓN**
>
> **Cupos disponibles** de un tour = cupo_maximo − suma de cantidad_personas de sus reservas en estado pendiente o confirmada. Las canceladas no ocupan cupo. Implemente este cálculo en el modelo Tour (por ejemplo, un método o un accessor) y reutilícelo en todas las vistas y validaciones.

04 Datos de prueba

El comando php artisan migrate:fresh --seed debe dejar la aplicación lista para usarse, sin pasos manuales. Como mínimo debe crear:

- Los roles admin, guia y turista (en un RoleSeeder).
- Usuarios fijos, todos con contraseña `password`: `admin@turipotosi.test`, `guia1@turipotosi.test`, `guia2@turipotosi.test`, `turista1@turipotosi.test`, `turista2@turipotosi.test`.
- Cinco categorías fijas: Histórico-cultural, Minero, Naturaleza, Gastronómico y Aventura.
- Seis servicios fijos: Transporte, Almuerzo, Entradas incluidas, Guía bilingüe, Seguro y Equipo de protección.
- Al menos 12 tours generados con TourFactory, distribuidos entre categorías y guías, cada uno con 2 a 4 servicios asociados en la pivote.
- Al menos 20 reservas generadas con ReservaFactory, de usuarios turistas, en distintos estados y sin superar el cupo de ningún tour.

05 Rutas mínimas

Puede agregar rutas, pero las siguientes deben existir con estos nombres. Agrupe las rutas por middleware y use prefijos donde corresponda.

Método	URI	Nombre	Acceso
GET	/	home	Público
GET	/tours	tours.index	Público
GET	/tours/{tour:slug}	tours.show	Público
POST	/tours/{tour}/reservas	reservas.store	turista
GET	/mis-reservas	reservas.mias	turista
PATCH	/reservas/{reserva}/cancelar	reservas.cancelar	turista (dueño)
GET	/guia/tours	guia.tours	guia
GET	/admin/tours (resource sin show)	admin.tours.*	admin
POST PUT DELETE	/admin/tours/...	store · update · destroy	admin
GET	/admin/reservas	admin.reservas.index	admin
PATCH	/admin/reservas/{reserva}/estado	admin.reservas.estado	admin

06 Requerimientos funcionales

Cada requerimiento indica su puntaje. Lea todos antes de empezar: varios dependen del modelo de datos, que conviene dejar bien hecho desde el primer día.

R1 · Modelo de datos y relaciones — 15 pts

- Migraciones de las cinco tablas con los tipos, claves foráneas y restricciones de la sección 3.
- Modelos con $fillable (o #[Fillable]), casts adecuados (precio, fecha_salida, activo) y todas las relaciones de la tabla de relaciones.
- Método o accessor de cupos disponibles en Tour, y un scope activos() que filtre tours activos con fecha_salida futura.

R2 · Factories y seeders — 10 pts

- TourFactory y ReservaFactory con datos verosímiles (títulos de tours reales de Potosí son bienvenidos).
- DatabaseSeeder que invoque los seeders en el orden correcto; migrate:fresh --seed debe ejecutarse sin errores en una base vacía.

R3 · Catálogo público — 15 pts

- Página de inicio con un hero y los 3 tours más próximos (scope activos, ordenados por fecha_salida).
- Listado `/tours` paginado de 9 en 9 que solo muestre tours activos, con filtro por categoría y búsqueda por título mediante query string (`?categoria=minero&buscar=cerro`). La paginación debe conservar los filtros (withQueryString).
- Detalle del tour por slug: imagen, descripción, precio, fecha, duración, categoría, guía, lista de servicios y cupos disponibles.
- Use eager loading (`with()`) para evitar el problema N+1 en listados.

R4 · CRUD de tours (admin) — 20 pts

- Controlador resource Admin\TourController con index (tabla paginada), create, store, edit, update y destroy.
- Validación en `StoreTourRequest` y `UpdateTourRequest`: título requerido y único, categoría existente, guía que exista y tenga rol guia, precio > 0, cupo entre 1 y 50, fecha_salida posterior a hoy, servicios como arreglo de IDs existentes.
- Selección de servicios con casillas de verificación, guardados con `sync()`. El formulario de edición debe mostrar marcados los servicios actuales.
- Slug generado automáticamente a partir del título. Mensajes de error junto a cada campo, old() para no perder datos y mensaje flash tras cada operación.
- Eliminar con confirmación. No se puede eliminar un tour que tenga reservas confirmadas: en ese caso se muestra un mensaje de error.

R5 · Imagen de portada — 10 pts

- Campo de imagen opcional en el formulario: jpg, png o webp, máximo 2 MB.
- Guardado en el disco `public` dentro de `tours/`, accesible mediante `storage:link`.
- Al reemplazar la imagen se borra la anterior del disco; al eliminar el tour también se borra su imagen.
- Si un tour no tiene imagen, las vistas muestran una imagen o bloque por defecto (placeholder).

R6 · Autenticación y autorización — 15 pts

- Registro, inicio y cierre de sesión. El registro asigna el rol turista.
- Rutas protegidas (role:admin, role:guia, role:turista). Un acceso indebido devuelve 403.
- Navegación que cambia según el rol con @role / @can: el admin ve el panel, el guía ve «Mis tours», el turista ve «Mis reservas».
- Página del guía: lista de sus tours (toursGuiados) y, por cada uno, las reservas con nombre del turista, cantidad de personas y estado.

R7 · Reservas — 10 pts

- Formulario de reserva en el detalle del tour (solo visible para turistas), con cantidad de personas y observación opcional.
- La validación impide reservar más personas que los cupos disponibles y reservar tours inactivos o con fecha pasada.
- Página «Mis reservas». Cancelar solo es posible si la reserva es propia y está pendiente: implemente una `ReservaPolicy` con el método `cancelar` y úsela con authorize() en el controlador y @can en la vista.
- Panel del admin con todas las reservas (filtrables por estado) y acción para cambiar el estado.

R8 · Calidad del proyecto — 5 pts

- Componentes Blade reutilizables como mínimo: `<x-layout>`, `<x-tour-card>` y `<x-flash>`. Diseño responsive con Tailwind v4.
- Historial de commits incremental y descriptivo (mínimo 15 commits repartidos en varios días) y README completo.

07 Entrega

1. Repositorio en GitHub llamado `INF560-EP2-Apellido`, público.

2. Sin `vendor/`, `node_modules/` ni `.env` en el repositorio. Debe incluir un `.env.example` configurado para PostgreSQL.
3. README con: descripción, requisitos, pasos de instalación, usuarios de prueba con su rol y al menos cuatro capturas de pantalla (catálogo, detalle, panel admin, mis reservas).
4. Enlace del repositorio subido a la tarea de Moodle antes de la fecha límite. Se evalúa el último commit anterior a esa hora.

El docente verificará cada proyecto ejecutando exactamente esta secuencia. Pruébela usted en una carpeta limpia antes de entregar:

verificacion.sh Copiar

```
# 1. Clonar e instalar dependencias
$ git clone https://github.com/<usuario>/INF560-EP2-Apellido.git
$ cd INF560-EP2-Apellido
$ composer install && npm install && npm run build

# 2. Configurar entorno (base de datos turipotosi en PostgreSQL)
$ cp .env.example .env && php artisan key:generate

# 3. Base de datos y almacenamiento
$ php artisan migrate:fresh --seed
$ php artisan storage:link

# 4. Ejecutar
$ php artisan serve
```

> **IMPORTANTE**
> Si el proyecto no se instala con esa secuencia, se descuentan 15 puntos y la revisión continúa con lo que funcione. La entrega fuera de plazo descuenta 10 puntos por día. Un repositorio con todo el código en uno o dos commits finales pierde los puntos de R8 y se revisa con especial atención en la defensa.

08 Defensa individual

Al ser un examen en casa, la nota depende de que usted demuestre que comprende y domina su propio código. La defensa dura entre 10 y 15 minutos, en su computadora, y tiene tres partes:

1. Demostración: ejecución de migrate:fresh --seed y recorrido de la aplicación con los tres roles.
2. Preguntas sobre su código: por qué está escrita una relación, una validación, una política o una consulta de esa manera.
3. Modificación en vivo: un cambio pequeño solicitado en el momento (por ejemplo, agregar un campo con su migración y validación, un nuevo filtro o una nueva regla en la Policy).

El puntaje del proyecto se multiplica por el factor de defensa:

Desempeño en la defensa	Factor
Explica con soltura su código y resuelve la modificación en vivo	1.0
Explica la mayor parte, con dificultades puntuales o la modificación incompleta	0.7
No logra explicar partes centrales del proyecto	0.4

Desempeño en la defensa	Factor
No se presenta a la defensa o no puede explicar su código	0.0

> **NOTA**
> Puede consultar la documentación oficial, las guías del curso y herramientas de asistencia, igual que en el trabajo profesional. Lo que no se negocia es la autoría: todo lo que entrega debe poder explicarlo y modificarlo usted mismo. Proyectos idénticos o con la misma estructura no justificada se califican con cero para todos los involucrados.

09 Rúbrica de evaluación

Criterio	Pts	Qué se verifica
R1 · Modelo de datos y relaciones	15	Migraciones, FK y restricciones; relaciones en los cinco modelos; cupos disponibles; scope activos
R2 · Factories y seeders	10	migrate:fresh --seed sin errores; roles, usuarios fijos, categorías, servicios, 12 tours, 20 reservas válidas
R3 · Catálogo público	15	Inicio; listado paginado con filtro y búsqueda que conserva query string; detalle por slug; eager loading
R4 · CRUD de tours	20	Resource controller; Form Requests con todas las reglas; sync de servicios; slug; flash; regla de eliminación
R5 · Imagen de portada	10	Validación de archivo; disco public; borrado al reemplazar y al eliminar; placeholder
R6 · Autenticación y autorización	15	Registro con rol turista; middleware de rol; 403; navegación por rol; vista del guía
R7 · Reservas	10	Validación de cupo y fecha; mis reservas; ReservaPolicy con authorize y @can; gestión de estados del admin
R8 · Calidad	5	Componentes Blade; responsive; commits incrementales; README completo
Total	**100**	× factor de defensa = nota final (más extras, hasta 100)

10 Plan de trabajo sugerido

Día	Avance sugerido
1	Crear el proyecto con PostgreSQL, instalar autenticación y spatie. Migraciones, modelos y relaciones (R1). Primeros commits.
2	Seeders y factories hasta que migrate:fresh --seed funcione sin errores (R2). Probar relaciones en Tinker.
3	Layout y componentes Blade; catálogo público con filtros y detalle (R3).
4	CRUD de tours con Form Requests, servicios e imagen (R4, R5).
5	Roles, middleware, navegación por rol y vista del guía (R6).
6	Reservas con validación de cupo y Policy (R7). Panel de reservas del admin.

Día	Avance sugerido
7	Pruebas en carpeta limpia con la secuencia de verificación, README, capturas y repaso de su propio código para la defensa.

EJEMPLO

Pregunta típica de defensa: «Si un turista cambia en el navegador el ID de la URL de cancelar, ¿qué impide que cancele la reserva de otra persona? Muéstreme la línea de código responsable.» Prepare respuestas de este tipo para cada requerimiento.